

5 Dataset and Experimental Setup

5.1 Datasets

Training problems. For RL we use `train_30k`, the 30,000-problem file that Hou et al. (2025) release with their training script. Each problem comes with a final answer and no reference solution. By source tag, it contains WebInstruct (26%), NuminaMath (24%) and its `cn_k12` subset (17%), Omni-MATH (13%), and smaller sources.

A tree whose leaves are all correct or all wrong yields zero advantage under tree-based credit, so it contributes no gradient. We therefore filter by difficulty. We sample 3,000 problems and let the base model answer each 8 times (temperature 1.0): 42% are never solved and 10% always.

We keep the 1,432 problems solved 1–7 times out of 8. Our long RL runs use 1,200 of them (150 steps $\times$ 8 problems), each seen once.

Inference-only study (Phase I). The EPTree study in Section 6.1 needs no training data. It uses 500 Omni-MATH problems (Gao et al., 2024), a random sample (seed 0) without any filtering, which is the same construction as the paper’s Omni-MATH-500. We chose hard, unfiltered problems because PassRate (at least one correct response) saturates on easier sets: an earlier run on 243 accuracy-filtered OlympiadBench problems reached 95% PassRate with 16 i.i.d. chains, leaving no room to compare samplers. On our Omni-MATH sample the base model solves 26% of problems with one response and 53% with 64.

Evaluation benchmarks. We evaluate on three sets, 1,082 problems in total:

- MATH500 (Hendrycks et al., 2021; Lightman et al., 2024): 500 problems.
- AMC: 83 problems from the AMC 12A/12B contests of 2022 and 2023.
- Omni-MATH-500: a random sample (seed 0) of 500 Omni-MATH problems (Gao et al., 2024). We score 499 of them; one prompt exceeds the 1,024-token limit.

Omni-MATH-500 overlaps `train_30k`: 14 of its problems, and 1 MATH500 problem, are among our 1,200 training problems. We report results with these 15 problems removed as well.

Reward and grading. The reward is binary: 1 if the last `\boxed{}` answer matches the reference under `math-verify`, else 0. A response cut off at the length limit counts as wrong.

5.2 Model and Training

Model. All experiments use Qwen2.5-Math-1.5B-Instruct (Yang et al., 2024) with its default chat template. We sample at temperature 1.0 and top- p 0.95. At the paper’s temperature of 1.2, this model produced degenerate output in 13% of samples. Responses are capped at 3,072 tokens for the Phase I study and 2,048 for RL; only 1.1% of the model’s responses exceed 2,048 tokens.

Training stack. We reuse the TreeRL authors’ released implementation unchanged for the learning algorithm itself: EPTree tree construction, tree-derived advantages and the policy-gradient loss. We port its infrastructure to a single 46 GB GPU:

- the policy (DeepSpeed ZeRO-2, CPU Adam) and the vLLM sampler share the GPU;
- updated weights reach vLLM through CUDA IPC instead of NCCL;
- the trees of all problems in a step are sampled in one batched vLLM call.

Peak GPU memory is 18.5–21 GB, so 1.5B-parameter RL fits comfortably in a 40 GB budget.

Methods compared. All three methods share the same code path and differ only in how responses are sampled and credited:

- **TreeRL:** EPTree with $(M, N, L, T) = (6, 2, 1, 2)$, i.e. 30 leaves per problem. 8 leaves are trained on, with TreeRL’s tree advantages (global plus local value differences, scaled by $1/\sqrt{|\mathcal{L}(v)|}$).
- **ChainRL:** 8 independent chains through the same code. Without branching, the tree advantage reduces to a leave-one-out (RLOO) baseline.
- **GRPO** (Shao et al., 2024): the same 8 chains with the group-normalised advantage $(r_i - \bar{r})/\text{std}(r)$. The KL penalty is set to $\beta = 0$, as for the other two methods.

Table 1 lists the settings next to the paper’s.

Evaluation protocol. We report greedy accuracy and `mean@8`, the accuracy averaged over 8 samples per problem at temperature 1.0. Because scoring the same model twice varies by about 0.5 points, models are compared on the same problems with a paired bootstrap: 2,000 resamples of problems, 95% CI.

6 Results

6.1 Phase I: surprisal-guided exploration

Phase I of our method builds the exploratory tree with EPTree’s surprisal criterion. We first replicate EPTree

	TreeRL paper	Ours
Policy	7–14B	Qwen2.5-Math-1.5B
RL steps	~300	40 / 150
Learning rate	1.5e-6	1.5e-6 / 5e-6
Problems / step	16	8
Trained resp./problem	30 Tree, 16 Chain	8
KL β	10^{-4}	0
Temperature, top- p	1.2, 0.95	1.0, 0.95
Max response tokens	8,192	2,048
Optimiser	ZeRO-3, CPU Adam	ZeRO-2, CPU Adam

Table 1: RL settings: the TreeRL paper and ours. Each of our steps uses 64 trained responses and one optimiser update.

as an inference-only study on 500 Omni-MATH problems (Section 5.1), using TreeRL’s own tree construction code. We compare three samplers:

- EPTree, which forks at the N highest-surprisal tokens (6 tree shapes);
- random forking with the same (M, N, L, T) , the paper’s ablation (4 of the shapes);
- i.i.d. chains: 64 per problem, from which $\text{pass}@k$ is estimated without bias for any $k \leq 64$.

Table 2 lists all eleven runs.

Trees are a cheaper way to search. Every tree run lies above the i.i.d. curve (Figure 1a). Measured as cost, the (6, 2, 1, 2) trees reach the PassRate of i.i.d. chains with 40% fewer tokens [15%, 57%]: 14.5k instead of 23.9k per problem. The (2, 3, 1, 1) trees save 36% [9%, 55%]; for the other matched shapes the savings are smaller and not significant (Table 2).

The saving does not come from better individual answers: mean per-response accuracy is 25.7–26.2% for every tree run, the same as a single chain (25.9%). It comes from sharing prefixes, which makes each additional response cheaper. Trees also give more distinct final answers per token: 8.4 for (6, 2, 1, 2), against about 7.0 for i.i.d. chains at the same budget.

The gain comes from hard problems. To see which problems benefit, we split them by how many of 32 held-out i.i.d. chains solve them, and compare the trees against the other 32 chains at the same token budget (Figure 1b). On the 256 problems that no held-out chain solves, the (6, 2, 1, 2) EPTree trees gain +3.3 points [+0.4, +6.5]. On problems solved sometimes, the gain is +4.3 [−2.0, +11.1]; with 119 problems this interval is wide. On problems solved by most chains it is 0, since both samplers are at ceiling. Trees help exactly where independent sampling rarely finds a solution.

Trees produce more learnable groups. In RL, a problem yields a gradient only when its responses include both a correct and an incorrect one. At the same

token budget, trees produce such mixed groups more often than i.i.d. chains (Figure 1c): 39.4% vs. 34.4% of problems for (6, 2, 1, 2), and 26.6% vs. 21.4% for (2, 3, 1, 1). This is a plausible mechanism for TreeRL’s advantage over ChainRL in Section 6.2.

Where to fork matters little for PassRate. At identical (M, N, L, T) , entropy-guided and random forking differ by +0.4, +0.8, 0.0 and −0.8 points, and every paired CI includes zero. The paper’s differences (+2.1, +1.0) lie inside our intervals. In this setting the efficiency gain comes from sharing prefixes, not from choosing the fork point.

Two of the paper’s descriptive results replicate. Fork positions are spread across the response (mean relative position 0.50). The most frequent fork tokens are function words and $\LaTeX$ delimiters (“the”, “(”, “and”); unlike the paper’s model, ours never writes “Wait”.

Surprisal does not predict consequential forks; position does. For credit assignment, the useful forks are those whose branch reaches a different outcome than the parent. Across all 168k forks, the share of branches that flip correctness is flat in the surprisal of the token forked at, between 5.6% and 7.5% from below 0.01 nats to above 8 nats (Figure 2a). It falls steadily with the fork’s position instead: from 9.7% for forks in the first fifth of a response to 3.4% in the last fifth (Figure 2b).

Comparing the two samplers directly, EPTree branches change the final answer more often than random branches (46.6% vs. 42.6%, paired +4.0 [+3.2, +4.8]). They change correctness only slightly more often (7.0% vs. 6.1%, +0.9 [+0.5, +1.3]). Over 90% of branches from either sampler reproduce their parent’s outcome.

Implications for our method. Surprisal is a reasonable signal for *diversity*: EPTree’s branches give more distinct answers. It is a weak signal for *consequentiality*, the property credit assignment needs. This supports our proposal in three ways:

- It confirms the limitation stated in Section 9: surprisal measures local uncertainty, not downstream usefulness.
- It motivates Phase II. Most rollouts spent on surprisal-chosen states produce no counterfactual evidence, so the reserved k rollouts should go where observed outcomes differ.
- It suggests position, or the remaining length after a state, as a cheap additional feature for the Phase II candidate set, which our ablations will test.

6.2 RL baselines

Short runs. With 40 steps at the paper’s learning rate, none of the three methods changes the model measurably. On greedy MATH500, for example, step-40

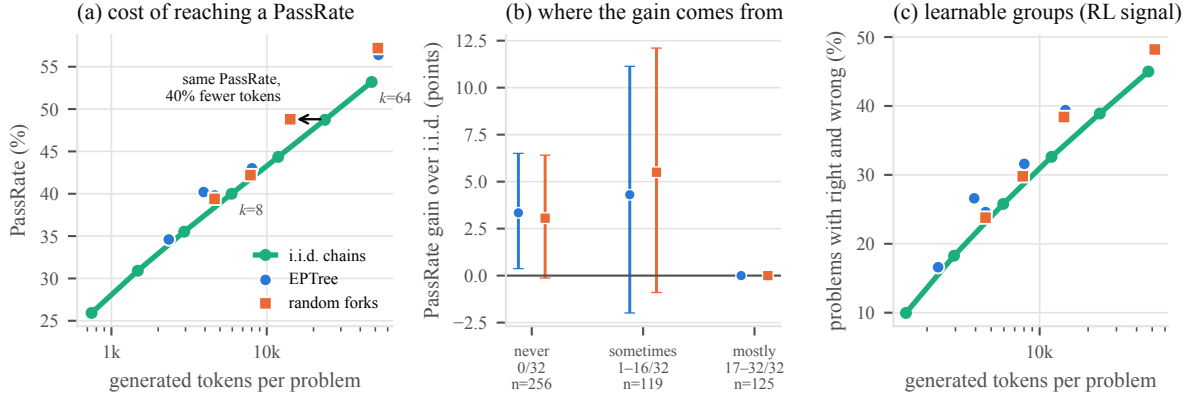


Figure 1: Tree sampling vs. i.i.d. chains on 500 Omni-MATH problems. (a) PassRate (at least one correct response) against generated tokens. The arrow marks the token cost i.i.d. chains need to match the (6, 2, 1, 2) tree. (b) Gain of the (6, 2, 1, 2) trees over i.i.d. chains at the same token budget, by problem difficulty, with 95% bootstrap CIs. Difficulty is how many of 32 held-out chains (chains 33–64) solve the problem. The i.i.d. reference uses the other 32 chains, so the two are independent. (c) Share of problems whose responses include both a correct and an incorrect one, i.e. a group with a learning signal for RL.

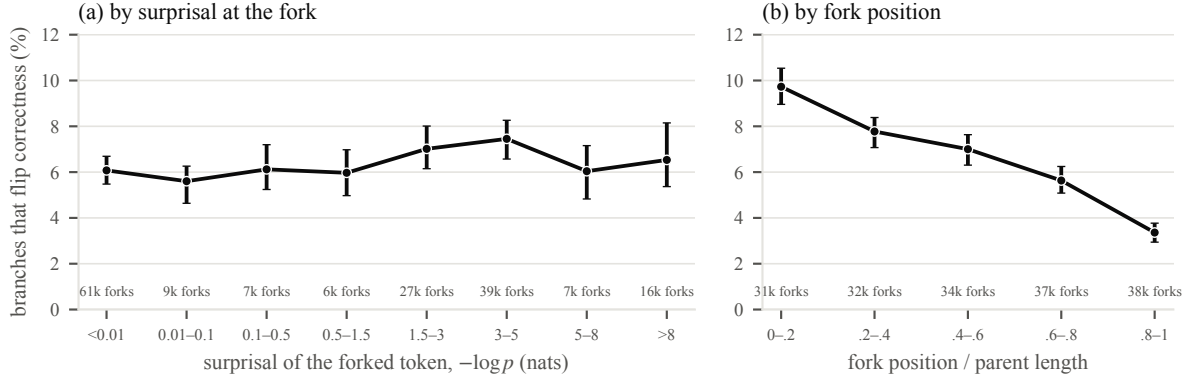


Figure 2: How often a branch changes correctness relative to its parent response, over all 168k forks of the ten tree runs (EPTree and random pooled). Error bars are 95% bootstrap CIs over trees. (a) By the surprisal of the token forked at. Random forks fill the low-surprisal bins, EPTree forks the high ones. (b) By the fork’s relative position in the parent response.

checkpoints score 74.4–75.4 against the base model’s 74.6, and every difference across the three benchmarks is within noise. We therefore ran 150 steps with a higher learning rate (5×10^{-6}).

Long runs. TreeRL and ChainRL are indistinguishable at steps 50 and 100 (Figure 3a). By step 150, TreeRL improves over the base model by +0.92 points [+0.17, +1.66], while ChainRL stays at the base level (−0.16 [−0.85, +0.52]).

Head-to-head on the same problems, TreeRL leads ChainRL by +1.09 [+0.32, +1.84] (Figure 3b, Table 3). It is ahead on every benchmark: MATH500 +1.07, AMC +2.41, Omni-MATH-500 +0.88. Excluding the 15 test problems that overlap the training data gives +1.04 [+0.27, +1.79].

The pattern matches the paper, where both methods

track each other early and TreeRL pulls ahead after about 100 of ~ 300 steps. The size of the gap is close to the paper’s for an already strong policy: +1.3 for R1-Distill-Qwen-7B.

Caveats.

- Each method was trained once, so run-to-run variance is not yet measured.
- The methods are matched on trained responses, not on generated tokens. Our TreeRL generates about $2.4 \times$ more tokens than ChainRL, whereas the paper matches generated tokens.
- Our budget (150 steps $\times$ 64 responses) is roughly 1/15 of the paper’s. The base policy, already tuned for math by its developers, leaves little headroom on MATH-level problems.

Sampler	(M, N, L, T)	Leaves	Tokens	PassRate	Gain [95% CI]	Tokens saved [95% CI]	Distinct	Mixed (%)
i.i.d.	$k = 4$	4	2.9k	35.5	—	—	2.51	18.3
i.i.d.	$k = 8$	8	5.9k	40.0	—	—	3.92	25.8
i.i.d.	$k = 16$	16	11.9k	44.4	—	—	6.00	32.6
i.i.d.	$k = 32$	32	23.7k	48.7	—	—	9.30	38.9
i.i.d.	$k = 64$	64	47.3k	53.2	—	—	14.41	45.0
EPTree	(2,1,1,1)	4	2.3k	34.6	+0.6 [−1.3, +2.6]	9% [−21, +34]	2.32	16.6
EPTree	(2,3,1,1)	8	3.9k	40.2	+2.8 [+0.6, +5.0]	36% [+9, +55]	3.56	26.6
EPTree	(4,1,1,1)	8	4.6k	39.8	+1.4 [−0.6, +3.5]	20% [−11, +43]	3.65	24.6
Random	(4,1,1,1)	8	4.6k	39.4	+1.0 [−1.0, +3.2]	15% [−16, +41]	3.63	23.8
EPTree	(4,3,1,1)	16	8.0k	43.0	+1.1 [−0.9, +3.2]	16% [−16, +42]	5.53	31.6
Random	(4,3,1,1)	16	7.9k	42.2	+0.4 [−1.6, +2.5]	7% [−28, +32]	5.38	29.8
EPTree	(6,2,1,2)	30	14.5k	48.8	+3.2 [+1.0, +5.3]	40% [+15, +57]	8.41	39.4
Random	(6,2,1,2)	30	14.2k	48.8	+3.3 [+1.2, +5.6]	41% [+18, +60]	8.07	38.4
EPTree	(8,4,2,2)	136	52.4k	56.4	—	—	21.07	48.4
Random	(8,4,2,2)	136	51.9k	57.2	—	—	19.92	48.2

Table 2: All Phase I runs on 500 Omni-MATH problems. *Tokens*: generated tokens per problem, counted once per shared prefix. *Gain*: PassRate minus i.i.d. pass@ k at the same token budget (interpolated log-linearly in tokens). *Tokens saved*: how many fewer tokens the tree uses than i.i.d. chains need to reach its PassRate. *Distinct*: distinct final answers per problem. *Mixed*: share of problems with at least one correct and one incorrect response. (8, 4, 2, 2) exceeds the 64-chain budget, so it has no matched i.i.d. comparison. EPTree minus Random at the same shape (paired, PassRate points): +0.4 [−2.0, +2.8], +0.8 [−1.6, +3.4], 0.0 [−2.4, +2.4], −0.8 [−3.2, +1.6]. Mean per-response accuracy is 25.7–26.2% for every tree run, against 25.9% for a single i.i.d. chain.

	MATH500	AMC	Omni	Pooled	Mean
Base	73.95	45.03	26.68	49.93	48.55
ChainRL	73.83	45.78	26.33	49.77	48.65
GRPO		in progress			
TreeRL	74.90	48.19	27.20	50.85	50.10
TreeRL − ChainRL, pooled: +1.09 [+0.32, +1.84]					

Table 3: Mean@8 accuracy (%) at step 150 of the long runs. Pooled: over all 1,082 problems. Mean: un-weighted mean of the three benchmarks, as the paper reports.

- The GRPO baseline has finished 50 of its 150 steps.

6.3 Summary of findings

- **Tree sampling is cheaper than i.i.d. sampling.** At the same PassRate, (6, 2, 1, 2) trees use 40% [15%, 57%] fewer tokens. The gain comes from shared prefixes, more distinct answers and more problems with mixed outcomes. It concentrates on problems that independent sampling rarely solves.
- **Fork placement does not matter for PassRate.** Entropy-guided forking does no better than random forking at equal tree shape (all paired CIs include zero).
- **Surprisal predicts diversity, not consequence.** The chance that a branch flips correctness is flat in surprisal but falls about $3\times$ from early to late forks, and over 90% of branches reproduce their parent’s outcome. This directly motivates our Phase II.

- **TreeRL beats ChainRL in RL, modestly.** With identical settings and budgets, TreeRL improves over ChainRL by +1.09 points [+0.32, +1.84] after 150 steps; the two are indistinguishable earlier. The gap is close to the paper’s for an already strong policy.
- **Single-GPU feasibility.** RL on a 1.5B model, with on-policy tree sampling, runs on one 46 GB GPU at a peak of 18.5–21 GB.

References

- Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. TreeRL: LLM reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s

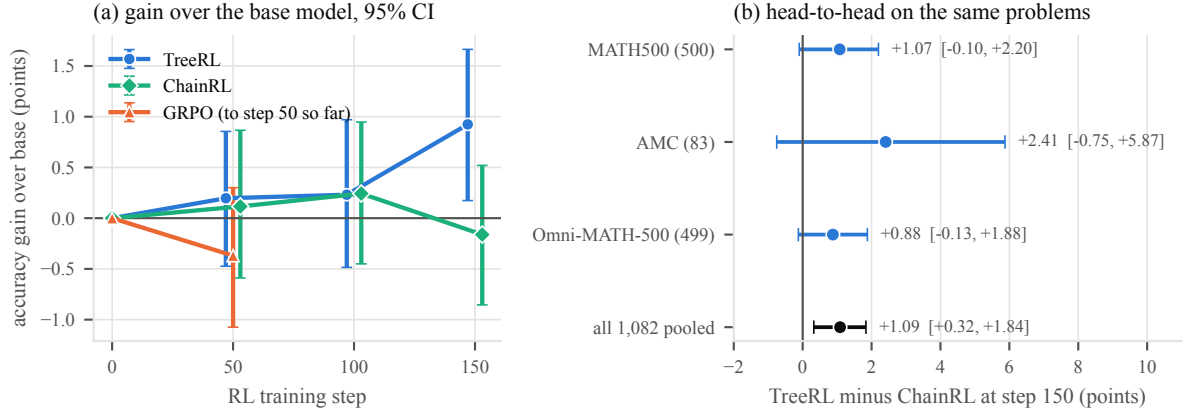


Figure 3: RL with Qwen2.5-Math-1.5B-Instruct: 150 steps, 8 problems $\times$ 8 trained responses per step. Accuracy is mean@8 on the same 1,082 test problems, with paired bootstrap 95% CIs. (a) Gain over the base model during training. GRPO is evaluated up to step 50 so far. (b) TreeRL minus ChainRL at step 150, per benchmark and pooled.

verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.